



UNIVERSIDADE FEDERAL DO CEARÁ

Code smells em frameworks front-end

Disciplina: Qualidade de Software

Equipe: Josias Batista e Guilherme Damasceno

Projeto: <https://github.com/appsmithorg/appsmith>

Fork: <https://github.com/josiasdev/appsmith>

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non Null Assertion

Minhas principais dificuldades na remoção do code smell são: Garantir que o código torne-se limpo sem quebrar o restante da aplicação através da criação do objeto.

Eu estou usando os seguintes métodos de refatoração para remover o code smell: Verificação de tipo através de um if anteriormente a criação do objeto para que possa verificar o tipo sempre que necessário e lançar um erro se for nulo.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non Null Assertion

Minhas principais dificuldades na remoção do code smell são: Avaliar os props que a função recebe para escolher a melhor forma de correção do code smell.

Eu estou usando os seguintes métodos de refatoração para remover o code smell: Atribuição obrigatório do tipo string no props e injetando default caso objeto seja nulo.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non Null Assertion

Minhas principais dificuldades na remoção do code smell são: Garantir que cada instância de Marking possa não ser undefined.

Eu estou usando os seguintes métodos de refatoração para remover o code smell: remoção do non null operator e alinhando interface de tipos de Making para garantir que alinhe com o contrato de tipos em runtime.



UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non Null Assertion

Minhas principais dificuldades na remoção do code smell são: Garantir verificação de tipo anteriormente ao non null operator para eliminar a chance de estado inválido durante produção.

Eu estou usando os seguintes métodos de refatoração para remover o code smell: introdução de Guard Clause com um if verificando a não existência de canvasWidget antes de utilizar seu valor.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non Null Assertion

Minhas principais dificuldades na remoção do code smell são: Verificar tipo anteriormente ao non null operator para eliminar chance de estado inválido e garantir que funcione dependendo do estado da ação customizada.

Eu estou usando os seguintes métodos de refatoração para remover o code smell: Introduzir um Guard Clause para verificação de tipo de duas variáveis e selecionar de acordo com ação para prosseguir com utilização do valor da variável selecionada.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non Null Assertion

Minhas principais dificuldades na remoção do code smell são: Garantir que o tipo nulo não seja válido e não afete o código em produção

Eu estou usando os seguintes métodos de refatoração para remover o code smell: criando um Guard Clause para verificação de valor de variável e retornando em caso nulo.



UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Multiple Booleans For State

Minhas principais dificuldades na remoção do code smell são: **Rastreamento de Dependências Espalhadas:** Identificar todos os locais onde os estados booleanos individuais (como `isDefaultSaving` ou `isPageNameSaving`) eram alterados, especialmente dentro de `useEffects` complexos que lidam com limpeza de estado (cleanup).

Sincronização de Ciclos de Vida: Ajustar a lógica de limpeza (reset) para garantir que o `savingField` retornasse a `null` no momento correto, dado que diferentes campos (como o "Page Name" e o "Set as Homepage") dependem de seletores Redux diferentes (`isPageLoading` vs `isUpdatingEntity`).

Garantia de Compatibilidade Visual: Substituir a lógica de estado sem quebrar a UI existente, assegurando que os ícones de carregamento (`TextLoaderIcon`) continuassem aparecendo nos locais corretos através do cálculo de estado derivado.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

State Unification (Unificação de Estado): Substituição de múltiplas variáveis de estado independentes (`useState(false)`) por uma única variável de referência (`savingField`) que armazena a chave do campo ativo.

Derived State (Estado Derivado): Criação de variáveis computadas (ex: `const isPageNameSaving = savingField === 'pageName'`) para manter a interface de leitura simples e evitar a necessidade de sincronizar múltiplos booleanos manualmente.

Single Source of Truth (Fonte Única de Verdade): Centralização da lógica de "o que está sendo salvo" em um único ponto, eliminando a possibilidade de estados impossíveis (como dois campos salvando simultaneamente).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Overly Flexible Props

Minhas principais dificuldades na remoção do code smell são:

Engenharia Reversa de Contratos Implícitos: Como o componente aceitava `any` ou espalhava props (`{...props}`) sem restrições, é difícil rastrear em todo o projeto quais propriedades estão realmente sendo passadas e utilizadas pelos componentes pais.

Limpeza de "Prop Drilling" Indesejado: Identificar quais propriedades estão sendo repassadas cegamente para elementos do DOM (como `<svg>` ou `<div>`) e causando avisos no console do React (ex: `Warning: React does not recognize the 'customProp' prop on a DOM element`).

Risco de Quebra Silenciosa: O receio de remover uma propriedade que parecia inútil, mas que estava sendo usada implicitamente por uma biblioteca de terceiros ou um hook interno que dependia dessa flexibilidade excessiva.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Explicit Interface Definition (Definição Explícita de Interface): Substituição do tipo `any`



UNIVERSIDADE FEDERAL DO CEARÁ

por interfaces TypeScript estritas (ex: `interface IconFallbackProps`) que documentam exatamente o que o componente espera receber.

Native Interface Extension (Extensão de Interfaces Nativas): Uso da herança de tipos do React (como `extends React.SVGProps<SVGSVGElement>`) para permitir todos os atributos HTML válidos padrão, mas bloquear propriedades inexistentes ou arbitrárias.

Props Whitelisting (Lista Branca de Propriedades): Em vez de espalhar tudo com `...props`, passei a desestruturar o objeto de propriedades para garantir que apenas os atributos seguros e validados sejam repassados para o elemento filho ou para o DOM.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Any Type

Minhas principais dificuldades na remoção do code smell são:

Engenharia Reversa de Tipos: O uso do `any` apaga o rastro do objeto original. A maior dificuldade é rastrear quem chama a função para descobrir qual é a estrutura real dos dados (ex: "Isso é um `HTMLInputElement` ou um `HTMLDivElement`?").

Ambiguidade em Eventos do DOM: No contexto de eventos (`SyntheticEvent`), o `any` mascara a origem do evento. É difícil saber quais propriedades (como `.value`, `.checked` ou `.dataset`) são seguras de acessar sem causar erros em tempo de execução.

Efeito Cascata de Erros: Frequentemente, remover um único `any` revela uma cadeia de inconsistências ocultas no código, pois outras partes do sistema dependiam dessa falta de validação para funcionar "na sorte".

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Strong Typing Adoption (Adoção de Tipagem Forte): Substituição da palavra-chave `any` por tipos concretos ou interfaces específicas (no nosso caso, `HTMLElement`), forçando o TypeScript a validar o código novamente.

Leveraging Library Generics (Uso de Genéricos da Biblioteca): Utilização correta dos recursos de tipagem do React (ex: `React.SyntheticEvent<T>`), definindo explicitamente o contrato do elemento que dispara o evento.

IntelliSense Restoration (Restauração do IntelliSense): O foco da refatoração é permitir que a IDE volte a funcionar, sugerindo propriedades válidas e alertando sobre acessos indevidos antes mesmo da compilação.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Enum Implicit Values

Minhas principais dificuldades na remoção do code smell são:

Compatibilidade com Dados Persistidos: O maior desafio é garantir que a mudança não quebre dados já salvos no banco de dados ou no `localStorage`. Se o sistema salvou o índice numérico `0` (implícito), e mudamos para uma string explícita



UNIVERSIDADE FEDERAL DO CEARÁ

"vertical", precisamos garantir que o sistema saiba lidar com essa migração.

Identificação de Dependências Ocultas: Encontrar locais no código que dependiam "acidentalmente" da ordem numérica do Enum (ex: loops que iteram baseados em índices ou lógica matemática como `if (direcao > 0)`).

Resistência à Mudança "Invisível": Justificar a refatoração de algo que "aparentemente funciona", explicando que a fragilidade está na manutenção futura (reordenação) e não no comportamento atual imediato.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Explicit String Assignment (Atribuição Explícita de Valores): Abandono do comportamento padrão do TypeScript (auto-incremento numérico) para atribuir valores constantes (ex: `VERTICAL = "vertical"`), tornando o Enum imutável independente de sua posição na lista.

Semantic Decoupling (Desacoplamento Semântico): Quebra do vínculo entre a "posição da linha no código" e o "valor real do dado", garantindo que inserir um novo item no topo da lista não altere os valores de todos os itens abaixo dele.

Improved Debugging Context (Melhoria no Contexto de Depuração): Adoção de **String Enums** para que logs e mensagens de erro exibam valores legíveis por humanos (ex: `"horizontal"`) em vez de "números mágicos" (ex: `1`), facilitando o rastreamento de bugs.